

Question-1

Definition :

1 Write a program to demonstrate conditional statements using if if-else and if-elif-else.

Aim/ Objective:

1. To understand the use of the `if` statement.
2. To understand how `if-else` is used to make decisions.
3. To understand the `if-elif-else` structure for multiple conditions.
4. To check conditions and execute the appropriate block of code.
5. To implement simple decision-making logic in Python.

Source Code:

#1 Write a program to demonstrate conditional statements using if if-else and if-elif-else.

```
#if
age=20

if age>=18:
    print("Adult")

#if else

n=20

if n%2==0:
    print("Odd")
else:
    print("Even")

#if elif else

mark=75
```

```
if mark >= 90:
    print("Grade : A")
elif mark >= 70:
    print("Grade : B")
elif mark >= 60:
    print("Grade : C")
else:
    print("Fail")
```

Output:

```
===== RESTART :
Adult
Odd
Grade : B
```

Conclusion :

The program successfully demonstrates conditional statements in Python. It shows how to use if, if-else, and if-elif-else statements to make decisions and display appropriate results based on given conditions.

Question-2**Definition :**

Write a program to check whether a number is positive negative or zero using nested conditions.

Aim/ Objective:

1. To understand the concept of nested `if` statements.
2. To accept a number from the user.

3. To check whether the number is greater than, equal to, or less than zero.
4. To display whether the given number is positive, negative, or zero.
5. To develop decision-making logic using Python conditional statements.

Source Code:

```
n=int(input("Enter a Number :"))

if n>=0:
    if n==0:
        print("Number Zero")
    else:
        print("Number Positive")
else:
    print("NUmber Negative")
```

Output:

```
===== RESTART:
Enter a Number :45
Number Positive
```

Conclusion:

The program successfully demonstrates nested conditions in Python. It checks the entered number and correctly identifies whether it is positive, negative, or zero using nested if statements.

Question-3**Definition :**

3. Write a program to generate a multiplication table using a for loop.

Aim/ Objective:

1. To understand the use of a `for` loop in Python.
2. To accept a number from the user.
3. To generate the multiplication table from 1 to 10.
4. To perform multiplication using the loop.
5. To display the multiplication table in a proper format.

Source Code:

```
n=int(input("Enter Number :"))
```

```
for i in range(1,11):  
    print(n,"x",i,"=",n*i)
```

Output:

```
===== RESTART: C:\`  
Enter Number :9  
9 x 1 = 9  
9 x 2 = 18  
9 x 3 = 27  
9 x 4 = 36  
9 x 5 = 45  
9 x 6 = 54  
9 x 7 = 63  
9 x 8 = 72  
9 x 9 = 81  
9 x 10 = 90
```

Conclusion:

The program successfully demonstrates the use of a for loop in Python. It generates and displays the multiplication table of the given number from 1 to 10.

Question-4**Definition :**

4. Write a program to find the sum of digits of a number using a while loop.

Aim/ Objective:

1. To understand the use of a `while` loop in Python.

2. To accept a number from the user.
3. To extract individual digits using the modulus (%) operator.
4. To calculate the sum of all digits.
5. To display the sum of digits of the given number.

Source Code:

```
'''
```

```
n=int(input("Enter Number :"))
```

```
sum=0
```

```
while n>0:
```

```
    m=n%10
```

```
    sum=sum+m
```

```
    num=n // 10
```

```
print("sum of =",sum)
```

```
'''
```

```
num = int(input("Enter a number: "))
```

```
sum = 0
```

```
while num > 0:
```

```
    digit = num % 10
```

```
    sum = sum + digit
```

```
    num = num // 10
```

```
print("Sum of digits =", sum)
```

Output:

```
===== RESTART:  
Enter a number: 987  
Sum of digits = 24
```

Conclusion:

The program successfully demonstrates the use of a while loop to find the sum of digits of a number. It repeatedly extracts each digit and adds it to calculate the final sum.

Question-5**Definition :**

5. Write a program to demonstrate the use of break continue and pass statements.

Aim/ Objective:

1. To understand the use of the `break` statement.
2. To understand the use of the `continue` statement.
3. To understand the use of the `pass` statement.
4. To learn how `break` stops a loop completely.
5. To learn how `continue` skips the current iteration.
6. To understand how `pass` is used as an empty statement.

Source Code:

```
for i in range(1,6):
```

```
if i==3:
    continue
if i==5:
    break
print(i)

if True:
    pass

print("Finish")
```

Output:

```
===== RESTART :
1
2
4
Finish
```

Conclusion:

The program successfully demonstrates the use of break, continue, and pass statements in Python. It shows how break terminates a loop, continue skips an iteration, and pass allows a statement block to remain empty.

Question-6**Definition :**

6. Write a program to iterate over lists strings and dictionaries using loops.

Aim/ Objective:

1. To understand how to iterate through a **list** using a **for** loop.
2. To understand how to iterate through characters of a **string**.
3. To understand how to iterate through **dictionary keys and values**.
4. To learn the use of the **for** loop with different Python data types.
5. To display the elements, characters, keys, and values of the given data.

Source Code:

```
# List
numbers = [10, 20, 30, 40]

print("List:")
for n in numbers:
    print(n)

# String
name = "Python"

print("String:")
for ch in name:
    print(ch)

# Dictionary
student = {
    "name": "Meet",
    "age": 21,
    "course": "Python"
}

print("Dictionary:")
for key, value in student.items():
    print(key, ":", value)
```

Output:

```
===== RESTART :
```

```
List:
```

```
10
```

```
20
```

```
30
```

```
40
```

```
String:
```

```
P
```

```
y
```

```
t
```

```
h
```

```
o
```

```
n
```

```
Dictionary:
```

```
name : Meet
```

```
age : 21
```

```
course : Python
```

Conclusion:

The program successfully demonstrates iteration over lists, strings, and dictionaries using loops in Python. It shows how a for loop can access each element or key-value pair in different data structures.

Question-7

Definition :

7. Write a program to demonstrate list dictionary and set comprehensions.

Aim/ Objective:

1. To understand the concept of **list comprehension**.
2. To create a list of **even number squares** using list comprehension.
3. To understand **set comprehension** and obtain unique word lengths.
4. To understand **dictionary comprehension**.
5. To create a dictionary that maps words to their **uppercase values**.

Source Code:

```
print("--- 1. List Comprehension (with condition) ---")
even_squares_list = [x * x for x in range(1, 11) if x % 2 == 0]
print(f"Even squares list: {even_squares_list}")
```

```
print("\n--- 2. Set Comprehension ---")
words = ["hello", "world", "python", "code", "loops", "code"]
unique_lengths = {len(word) for word in words}
print(f"Unique word lengths: {unique_lengths}")
```

```
print("\n--- 3. Dictionary Comprehension ---")
word_dict = {word: word.upper() for word in words}
print(f"Word mapping dictionary:\n{word_dict}")
```

Output:

```
===== RESTART: C:\Users\student\Downloads\Unit_2_python\p_7.py =====
--- 1. List Comprehension (with condition) ---
Even squares list: [4, 16, 36, 64, 100]

--- 2. Set Comprehension ---
Unique word lengths: {4, 5, 6}

--- 3. Dictionary Comprehension ---
Word mapping dictionary:
{'hello': 'HELLO', 'world': 'WORLD', 'python': 'PYTHON', 'code': 'CODE', 'loops'
: 'LOOPS'}
```

Conclusion:

The program successfully demonstrates list, dictionary, and set comprehensions in Python. It shows how comprehensions can be used to create collections in a concise and simple way.

Question-8

Definition :

8. Write a program to illustrate variable scope using local global and nonlocal variables.

Aim/ Objective:

1. To understand the concept of variable scope in Python.
2. To understand the use of local variables inside a function.
3. To understand the use of global variables defined outside functions.
4. To understand the use of the `nonlocal` keyword in nested functions.
5. To understand how the `global` keyword modifies a global variable.

Source Code:

```
# 1. Global Variable: Defined outside any function
name = "Global"
```

```
def outer_function():
    # 2. Local Variable: Defined inside a function (acts as enclosing scope for inner_function)
    name = "Local (Outer Function)"
    print(f"Inside outer_function (before inner): {name}")

    def inner_function():
        # 3. Nonlocal Keyword: Modifies the variable in the nearest enclosing scope
        (outer_function)
        nonlocal name
        name = "Nonlocal (Modified by Inner Function)"
        print(f"Inside inner_function: {name}")

    inner_function()
    # The value here is changed because of the 'nonlocal' keyword inside inner_function
    print(f"Inside outer_function (after inner): {name}")

def modify_global():
    # Global Keyword: Tells Python we want to modify the global variable, not create a local one
    global name
    name = "Global (Modified)"
    print(f"\nInside modify_global: {name}")

# Execution starts here
print(f"Starting value: {name}\n")

print("--- Testing Local and Nonlocal ---")
outer_function()

print("\n--- Testing Global ---")
print(f"Global value before modify_global: {name}")
modify_global()
print(f"Global value after modify_global: {name}")
```

Output:

```
===== RESTART: C:\Users\student\Downloads\Unit_2_python\p_8.py =====
Starting value: Global

--- Testing Local and Nonlocal ---
Inside outer_function (before inner): Local (Outer Function)
Inside inner_function: Nonlocal (Modified by Inner Function)
Inside outer_function (after inner): Nonlocal (Modified by Inner Function)

--- Testing Global ---
Global value before modify_global: Global

Inside modify_global: Global (Modified)
Global value after modify_global: Global (Modified)
|
```

Conclusion:

The program successfully demonstrates local, global, and nonlocal variable scopes in Python. It shows how variables can be accessed and modified according to their scope within functions and nested functions.

Question-9

Definition :

9. Write a program to demonstrate iterators and iterables in Python.

Aim/ Objective:

1. To understand the concept of an **iterable** in Python.
2. To create an **iterator** from a list using the `iter()` function.
3. To retrieve elements from an iterator using the `next()` function.
4. To understand how an iterator moves through elements one by one.
5. To understand the **StopIteration exception** when there are no more elements.

Source Code:

```
print("--- 1. The Iterable ---")
my_list = ["Apple", "Banana", "Cherry"]
print(f"My list: {my_list}")

print("\n--- 2. Creating the Iterator ---")
my_iterator = iter(my_list)
print("Successfully created an iterator from the list.")

print("\n--- 3. Stepping through with next() ---")
print(f"First item: {next(my_iterator)}")
print(f"Second item: {next(my_iterator)}")
print(f"Third item: {next(my_iterator)}")

print("\n--- 4. Hitting the end ---")
try:
    print(next(my_iterator))
except StopIteration:
    print("Caught a StopIteration! There are no more items left to fetch.")
```

Output:

```
===== RESTART: C:\Users\student\Downloads\Unit_2_python\p_9.py
--- 1. The Iterable ---
My list: ['Apple', 'Banana', 'Cherry']

--- 2. Creating the Iterator ---
Successfully created an iterator from the list.

--- 3. Stepping through with next() ---
First item: Apple
Second item: Banana
Third item: Cherry

--- 4. Hitting the end ---
Caught a StopIteration! There are no more items left to fetch.
```

Conclusion:

The program successfully demonstrates iterables and iterators in Python. It shows how a list can be iterated using a for loop and how an iterator can be created using iter() and accessed using next().

Question-10

Definition :

10. Write a program to generate a sequence of numbers using generator functions and yield keyword.

Aim/ Objective:

1. To understand the concept of **generator functions** in Python.
2. To understand the use of the **yield keyword**.
3. To generate even numbers up to a given limit.
4. To retrieve values from a generator one at a time.
5. To understand how generators efficiently produce a sequence of values.

Source Code:

```
def generate_evens(limit):
    print("[Generator Started]")
    number = 2

    while number <= limit:
        yield number
        number += 2

    print("[Generator Finished]")

print("--- Creating the Generator ---")
even_sequence = generate_evens(8)
print("Generator object created successfully.")

print("\n--- Fetching Numbers from the Generator ---")
for num in even_sequence:
    print(f"Received: {num}")
```

Output:

```
===== RESTART: C:\Users\student\Downloa
--- Creating the Generator ---
Generator object created successfully.

--- Fetching Numbers from the Generator ---
[Generator Started]
Received: 2
Received: 4
Received: 6
Received: 8
[Generator Finished]
|
```

Conclusion:

The program successfully demonstrates the use of a generator function and the yield keyword in Python. It generates a sequence of numbers one at a time and displays the generated values using a for loop.